面向边缘 AI 大语言模型推理的 RISC-V 自定义 vdot 指令设计——基于“香山”昆明湖 V2 处理器架构

第三阶段竞赛报告

NEMU 协同仿真与性能分析

目标平台 香山昆明湖 V2，VLEN=128

队伍 群贤芯跳

目录

一 阶段目标与完成情况 3

二 NEMU 参考模型实现 4

三 协同仿真与功能验证 7

四 基线构建与性能分析 10

五 阶段成果与后续完善 12

一 阶段目标与完成情况

我们为 NEMU 增加了 vdot 执行模型，在香山仿真中启用 Difftest，完成固定 e8 点积语义的定向协同验证。同时采用既有 RVV 指令实现点积基线，建立 baseline、exact 和 empty 三份计时程序，获得可复算的周期比较结果。

本阶段完成参考模型接入、RTL 与 NEMU 定向协同仿真、RVV 基线构建及片段周期分析，对应赛题第三阶段的功能与性能要求。

交付内容包括 NEMU 注册、译码与 helper 实现，七组 exact 系统用例，RVV baseline 功能程序，以及三份计时夹具的源码和原始日志。

定向协同运行输出 S00000000，日志记录参考模型加载与 Difftest enabled，未出现 different at pc；基线功能程序输出 S0。单次迭代夹具扣除 empty 开销后，baseline 为 1255 cycles，exact 为 53 cycles，片段周期比为 23.679。

我们以同一份八项有符号运算定义连接硬件、参考模型和测试软件，并同时保留程序自检、Difftest 记录及原始周期标记。三类材料共同支撑了功能结果和样本性能分析。

证据层次	当前可确认的内容	本次验证范围
NEMU 源码	注册、译码和 helper 与本次 e8 定义对应	固定八项 e8 运算契约
系统定向程序	七组结果及高位清零比较后输出成功串	七组输入，vm=1、vl=8、vstart=0
协同仿真日志	Difftest enabled，参考模型路径和成功串可查	成功标记前的定向运行；随后人工停止
性能记录	一次迭代、该仿真配置和夹具的周期观察	当前 emu 的一次迭代夹具

二 NEMU 参考模型实现

2.1 软硬件运算定义

我们将第二阶段的输入与结果格式落实到参考模型：VLEN=128，源元素为 e8，运算固定取两个源的低八个 byte；归约结果以二补码写入低 32 位，高 96 位清零。

```
acc = 0
for k in 0..7:
    acc += signed8(vs1[8*k +: 8]) * signed8(vs2[8*k +: 8])
vd[31:0] = acc[31:0]
vd[127:32] = 0
```

该定义覆盖的八项和范围为 -130048 至 131072，32 位结果可完整表示。系统用例分别检查八项 1×2 的结果 16、八项 -128×1 的结果 -1024，以及正负边界和混合符号结果。

实现位置	对应代码或行为	一致性要点
Chisel VDotU.scala	.asSInt、8 路 products、扩位归约、Cat(0, result32)	取低 8 个 byte；不读取旧 vd
NEMU vcompute.h	get_vreg → (int8_t) → acc → set_vreg	同样恢复 signed int8 语义并清零高位
exact 汇编	vsetvli e8；VDOT；改为 e32 后存回四个 word	检查低 word 及另外三个 word 为零

本次系统验证采用 LMUL=1、vl=8、vm=1、vstart=0。helper 按固定八次循环实现该运算契约；mask、其他 vl/vstart 和 LMUL 组合列为后续语义扩展与验证内容。

2.2 helper 注册与译码

我们完成了 helper 注册、编码译码和执行函数三处接入，使测试程序发出的自定义机器码能够在 NEMU 中进入 vdot 状态更新逻辑。

文件 路径以 coding 为根	修改点	实现内容
overlay/NEMU/src/isa/riscv64/include/isa-all-instr.h	helper 列表新增 f(vdot)	生成的执行分发中包含 vdot 名称
overlay/NEMU/src/isa/riscv64/instr/rvv/decode.h	OP_V 层和 vopmvv 表内均有 vdot 模式	funct6=101100；本次 VV 的 funct3=010
overlay/NEMU/src/isa/riscv64/instr/rvv/vcompute.h	def_EHelper(vdot)	向量条件检查、读取、计算、写回和状态更新

```
// isa-all-instr.h 的指令 helper 列表节选
f(vmadd) f(vnmsub) f(vmacc) f(vnmsac) f(vdot) f(vwaddu)

// decode.h 的 OP_V 入口，位于分组分发之前
def_INSTR_TAB("101100 ? ????? ????? 010 ????? ????? ??", vdot);
```

译码在 OP_V 向量指令空间内限定 funct6 与 funct3，寄存器号和 vm 字段由模式中的可变位接收。源码在 OP_V 与 vopmvv 两层保留对应匹配，两者指向同一 vdot 执行功能。

我们在汇编中采用 .word 构造自定义机器码，宏参数顺序为 vd、vs1、vs2，对应字段 vs2[24:20] 与 vs1[19:15]。该方式使系统测试能够使用现有交叉工具链构建。

```
.macro VDOT vd, vs1, vs2
  .word ((0x59 << 25) | (\vs2 << 20) | (\vs1 << 15) | (2 << 12) | (\vd << 7) | 0x57)
.endm
# 测试程序调用 VDOT 8, 6, 4 ；编码为 0xB2432457
```

2.3 执行与状态更新

NEMU vdot helper 当前交付源码

vcompute.h 八项 signed int8 归约 低 32 位写回 高 96 位清零

```
873  def_EHelper(vdot) {
874    require_vector(true);
875    if (vtype->vsew != 0) {
876      longjmp_exception(EX_II);
877    }
878    int vlmul = vtype->vlmul;
879    uint32_t acc = 0;
880    for (int k = 0; k < 8; k ++) {
881      rtlreg_t a = 0, b = 0;
882      get_vreg(id_src->reg, k, &a, 0, vlmul, 1, 1);
883      get_vreg(id_src2->reg, k, &b, 0, vlmul, 1, 1);
884      acc += (uint32_t)((int32_t)(int8_t)a * (int32_t)(int8_t)b);
885    }
886    set_vreg(id_dest->reg, 0, (rtlreg_t)acc, 2, vlmul, 1);
887    for (int idx = 4; idx < 16; idx ++) {
888      set_vreg(id_dest->reg, idx, 0, 0, vlmul, 1);
889    }
890    vp_set_dirty();
891    vstart->val = 0;
892  }
```

图为源码清单整理，并非仿真波形或新运行截图。原始完整文件在 coding/overlay/NEMU。

图 1 当前交付 vcompute.h 中的完整 vdot helper 源码清单

NEMU helper 按向量状态检查、源 byte 读取、有符号乘法、低 word 写回和高位清零组织实现，并在完成后更新向量状态。下面给出对应的关键代码。

向量条件检查

```
require_vector(true);
if (vtype->vsew != 0) {
  longjmp_exception(EX_II);
}
```

helper 通过 require_vector 检查向量执行条件，并以 vsew==0 对应 e8；非 e8 进入 EX_II 路径。当前七组系统用例验证 e8 运算，非 e8 trap 的系统观察列为后续异常验证项。

从字节读取到有符号乘法

```
rtlreg_t a = 0, b = 0;
get_vreg(id_src->reg,  k, &a, 0, vlmul, 1, 1);
get_vreg(id_src2->reg, k, &b, 0, vlmul, 1, 1);
acc += (uint32_t)((int32_t)(int8_t)a * (int32_t)(int8_t)b);
```

我们使用 get_vreg 逐 byte 读取两个源，将每项先转为 int8_t，再扩展到 int32_t 参与乘法。由此保证 0x80 按 -128 运算，与 Chisel 中的 .asSInt 一致。

acc 使用 uint32_t 保存 32 位二补码位模式，负乘积按 32 位模值累加。由于八项和处于有符号 32 位范围内，最终按 int32 解释即可得到预期的正负结果。

写回和向量状态

```
set_vreg(id_dest->reg, 0, (rtlreg_t)acc, 2, vlmul, 1);
for (int idx = 4; idx < 16; idx ++) {
  set_vreg(id_dest->reg, idx, 0, 0, vlmul, 1);
}
vp_set_dirty();
vstart->val = 0;
```

结果写回采用宽度参数 2 的 set_vreg 保存低 32 位，再以 byte 宽度清零索引 4 至 15。完成后调用 vp_set_dirty，并将 vstart 清零，实现运算结果与向量状态的同步更新。

字节区间	位段	写回后含义
byte 0–3	vd[31:0]	完整 32 位结果，包括负数的二补码高字节
byte 4–15	vd[127:32]	固定为零

我们将高位清零的起始位置限定为 byte 4，完整保留结果的 byte 0 至 3。该边界使 -1024 的 0xFFFFFC00 等负结果保持完整，同时满足高 96 位清零要求。

三 协同仿真与功能验证

3.1 协同验证结果

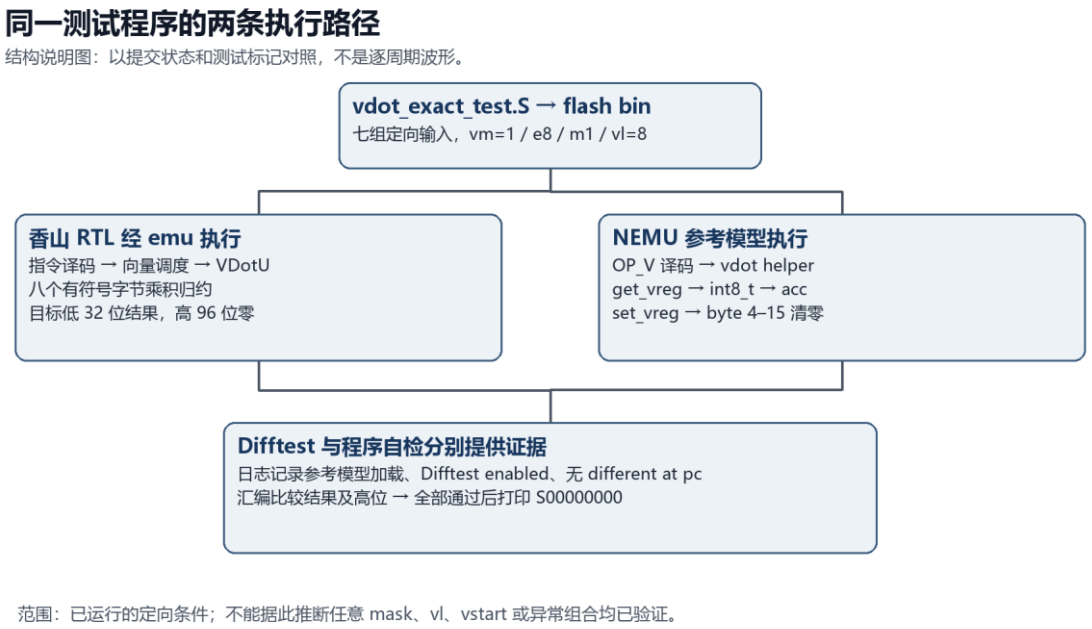


图 2 香山与 NEMU 的执行路径以及日志和程序标记的关系

我们建立了两层功能检查：系统程序将向量结果存入内存并逐 word 自检，Difftest 对照香山与 NEMU 的执行状态。前者验证结果格式，后者检查软硬件模型的一致性。

```
The reference model ... /NEMU/build/riscv64-nemu-interpreter-so
The first instruction of core 0 has commited. Difftest enabled.
... Read 1544 bytes from file ../vdot_exact_test.bin ...
S00000000
```

本次协同日志保存于 vdot_exact_cosim_final.log，包含参考模型路径、Difftest enabled 和 S00000000 成功标记；该记录未报告不同 PC 状态。程序自检与参考模型对照共同验证了这次定向运行。

测试程序按顺序检查七组用例，全部通过后输出固定标记 S00000000；失败路径输出 F 与用例编号。我们据此将成功标记与用例表关联，检查范围以实际执行的七组输入为准。

3.2 七组系统测试

我们在 RUN_CASE 中完成源数据装载、目的寄存器预置、e8 点积执行和完整结果存回：先用 e64 装载打包数据，再以 e32 将目的四个 word 预置为 7，执行 e8 VDOT 后切回 e32 保存全部 128 位结果。

```
vsetvli zero, t0, e32, m1, tu, mu
vmv.v.i v8, 7           # 先放入非零旧值
...
vsetvli zero, t0, e8, m1, tu, mu
VDOT 8, 6, 4
...
vsetvli zero, t0, e32, m1, tu, mu
vse32.v v8, (a0)
lw t1, 0(a0)            # 比较低 32 位
...
lw t1, 4(a0)            # 另外三个 word 均要求为 0
```

目的 v8 的四个 word 预置为 7，使测试同时检查旧值覆盖与高位清零。执行后低 word 为期望点积，另外三个 word 为零，符合本设计不将旧 vd 加入点积的语义。

编号	输入含义	有符号结果	低 32 位
0	八项 1×2	16	00000010
1	八项 -128×1	-1024	FFFFFC00
2	-128×1 + 127×1 + -1×1；其余为零	-2	FFFFFFFE
3	八项 127×127	129032	0001F808
4	八项 -128×-128	131072	00020000
5	八项 -128×127	-130048	FFFE0400
6	八项 0×127	0	00000000

每组用例通过有符号 lw 比较低 word，并分别检查偏移 4、8、12 的高 word 为零。七组覆盖正数、负数、混合符号、最大正和、最小和及零值方向。

3.3 数据通路波形验证

当前 e8 独立组合 RTL 的 VCD 数值图

从真实 vdot_exact_wave.vcd 提取；每格为一组输入的保持区间，时间单位 ns

	0–1 ns	1–2 ns	2–3 ns	3–4 ns	4–5 ns
vs1[63:0]	0101010101010101	8080808080808080	0000000001FF0201	7F7F7F7F7F7F7F7F	8080808080808080
vs2[63:0]	0202020202020202	0101010101010101	000000000103FF02	7F7F7F7F7F7F7F7F	7F7F7F7F7F7F7F7F
vd[31:0]	00000010	FFFFFC00	FFFFFFFE	0001F808	FFFE0400
vd[127:32]	000000…0000	000000…0000	000000…0000	000000…0000	000000…0000
expected[31:0]	00000010	FFFFFC00	FFFFFFFE	0001F808	FFFE0400

对应结果：16　-1024　-2　129032　-130048

该文件没有 clk、reset、valid 或 ready。#1 是 testbench 的观测间隔，不是 CPU 指令延迟。

图 3 当前 e8 独立组合模型 VCD 的数值整理 详见原始 VCD

独立组合 VCD 验证了 e8 数据通路的输入输出：0—1 ns 激励为八项 1×2，低 word 输出 0x00000010；1—2 ns 的八项 -128×1 输出 0xFFFFFC00。各组高 96 位均为零，vd 与 expected 一致。

该波形记录独立组合 testbench 的数值响应，1 ns 为每组激励的观察间隔。核级协同结果由 Difftest 日志提供，单元流水延迟的测量属于后续集成波形工作。

```
gtkwave <实际路径>/coding/evidence/waveforms/vdot_exact_wave.vcd
# 展开 tb_vdot_exact
# 选择 vs1、vs2、vd、check.expected、checks、errors
# 总线设置 Hexadecimal；Zoom Fit；移动光标逐组核对
```

我们将 VDotU.scala、vcompute.h 与独立 VCD 按源 byte 读取、有符号乘法、归约和高位清零逐项对应，形成软硬件数据语义的分析材料。

3.4 系统运行流程

我们将参考模型、系统汇编、链接脚本和执行入口一并整理到 coding 目录。实际协同运行使用 VM 的 /home/wyc/Desktop/vdot-xs 工程，流程依次为生成 NEMU 动态库、构建测试镜像和带参考模型启动香山 emu。

```
cd ~/Desktop/vdot-xs/NEMU
export NEMU_HOME=$PWD
make -B -j8

cd ~/Desktop/vdot-xs
riscv64-linux-gnu-gcc -march=rv64gcv -mabi=lp64d \
  -fno-pic -mcmodel=medany -nostdlib -nostartfiles \
  -T flash.ld vdot_exact_test.S -o vdot_exact_test.elf
riscv64-linux-gnu-objcopy -O binary \
  vdot_exact_test.elf vdot_exact_test.bin

cd XiangShan
./build_exact/emu -F ../vdot_exact_test.bin \
  --diff=../NEMU/build/riscv64-nemu-interpreter-so -I 200000
```

构建流程将 NEMU_HOME 指向 NEMU 根目录，并使用 make -B 刷新注册、译码和 helper 对应的构建产物。实际运行所加载的参考模型由协同日志中的动态库路径和启用信息记录；构建过程日志另存于附件。

系统程序在输出成功标记后进入自循环。我们在观测到完整标记后停止仿真，并在复核日志中记录了该终止方式；功能结论依据标记前完成的七组自检与 Difftest 记录。

我们为系统复现脚本加入 GNU timeout、有界等待、成功标记与 mismatch 检查，并完成 bash -n 语法检查。本次协同结果来自保存的单项运行记录，脚本端到端批量回归列为后续完善内容。

四 基线构建与性能分析

4.1 RVV 基线实现

我们以既有 RVV 指令构建点积基线，配置为 e32/m1、vl=2，将八个 byte 打包到两个 32 位元素中。四轮移位完成 byte 符号扩展，vmacc 累加各位置乘积，再由 vredsum 归约两组部分和。

```
vsll.vi v12,v4,24
vsra.vi v12,v12,24       # 源 1 的 byte 0 符号扩展
vsll.vi v13,v6,24
vsra.vi v13,v13,24       # 源 2 的 byte 0 符号扩展
vmacc.vv v8,v12,v13
# 另外三轮左移量为 16、8、0，均算术右移 24
...
vredsum.vs v24,v8,v16    # v16[0]=0，归约两个 32 位元素
```

步骤	32 位组内的结果	两组的最终关系
4 轮移位加 vmacc	每个元素累加它包含的四对 byte 乘积	v8[0] 与 v8[1] 共包含八项
vredsum.vs	将 v8 两元素求和，以 v16[0]=0 起始	结果放在 v24 的第 0 个元素
vmv.x.s 与比较	取出结果并与 16 比较	功能程序正确时打印 S0

基线功能程序 vdot_baseline_supported.S 在八项 1×2 输入下得到 16，并输出 S0。性能测量使用独立的 vdot_baseline_full1.S 夹具记录周期；多组负数、边界及随机输入的基线等价性验证列为后续扩展。

4.2 计时方法与夹具



图 4 源码中的计时区间 UART 输出位于两次 rdcycle 之后

我们为 baseline、exact 和 empty 配置一次迭代的计时流程：两次 rdcycle 分别保存起止值，s3=s2-s1。输入准备位于计时区间前，结果提取和 UART 输出位于区间后；baseline 采用 e32/vl=2，exact 采用 e8/vl=8。

```
# baseline 计时区间
rdcycle s1
.L:
  vmv.v.i v8,0
  ...                 # 16 条移位 + 4 条 vmacc
  vredsum.vs v24,v8,v16
  addi t5,t5,-1
  bnez t5,.L
rdcycle s2
sub s3,s2,s1
# 后面才打印 B 和四位十六进制数
```

```
# exact 计时区间          # empty 计时区间
rdcycle s1               rdcycle s1
.L:                      .L:
  VDOT 8,6,4               addi t5,t5,-1
  addi t5,t5,-1            bnez t5,.L
  bnez t5,.L             rdcycle s2
rdcycle s2                sub s3,s2,s1
sub s3,s2,s1
```

基线循环包含累加器清零、16 条移位、4 条 vmacc、1 条归约及循环控制；exact 循环以一条 VDOT 和循环控制完成对应计算。片段周期差体现了指令序列、数据依赖和调度开销的综合变化。

我们设置 empty 夹具估计读计数器、循环控制与相关流水开销。该夹具沿用 exact 的前置配置，按相同单迭代方式测得 148 cycles，并用于本次样本的近似开销扣除。

4.3 周期结果与加速比

源码和保存日志	原始标记	十进制原始值
vdot_baseline_full1.S / .log	B057B	0x057B = 1403
vdot_exact_full1.S / .log	E00C9	0x00C9 = 201
vdot_empty_match1.S / .log	O0094	0x0094 = 148

性能程序以 B、E、O 区分三类片段，并由 hex4 输出 s3 的低 16 位。本次原始周期值分别为 1403、201、148，均在四位十六进制显示范围内。

```
baseline_net = 1403 - 148 = 1255 cycles
exact_net    =  201 - 148 =   53 cycles
ratio        = 1255 / 53 = 23.679245...
```

比较口径	数值	如何理解
原始片段周期比	1403 / 201 ≈ 6.980	不做空循环扣除的记录比值
按夹具扣除后的比值	1255 / 53 ≈ 23.679	单次迭代、相同模型的扣除结果
对外表述	约 23.679 倍	单次迭代与当前夹具条件

我们同时给出原始周期比 6.980 与扣除后周期比 23.679，公开 empty 扣除对结果的影响。该结果对应一次迭代、当前 emu 配置和上述夹具，描述的是测量片段的样本周期比较。

本次计时采用两次 rdcycle 的边界，结果读取位于区间之后，未设置显式向量完成同步。因此本阶段以片段周期比报告性能结果；依赖链延迟、稳定吞吐、多次运行与完整计数输出列为后续测量工作。

五 阶段成果与后续完善

我们实现了 NEMU 的 vdot 注册、译码与执行逻辑，完成七组系统定向协同检查，并构建了 RVV 基线和三份计时夹具。功能验证记录了成功标记与参考模型对照；性能样本扣除 empty 后为 1255 与 53 cycles。后续完善重点如下。

事项	当前已经有的证据	仍应补充的工作
有符号及结果格式	七组 exact 系统用例、独立组合 RTL 五组	随机数据、源高字节不参与、目的别名用例
软硬件一致性	参考模型加载、Difftest enabled、成功串	更长程序、更多依赖和异常交互
mask / vl / vstart	本次固定 vm=1、vl=8、vstart=0	明确规范并检查未测参数；当前代码不做通用掩码合并
非法 SEW	源码包含 e8 合法性判断	系统级 trap / exception 观察
baseline 功能	保存的单组输入输出 S0	负数、极值及随机等价性比较
性能	三份单迭代原始标记	多次运行、更多迭代、同步边界及原始完整计数

我们完成了 NEMU 参考模型接入、七组系统定向协同验证、RVV 基线实现和单次片段性能比较。项目交付了源码、程序、原始日志和计算过程，后续将围绕参数覆盖、基线等价性与统计测量进一步完善。

附录 交付文件索引

提交目录内的文件	用途
coding/overlay/NEMU/src/isa/riscv64/include/isa-all-instr.h	helper 注册
coding/overlay/NEMU/src/isa/riscv64/instr/rvv/decode.h	OP_V / vopmvv 译码
coding/overlay/NEMU/src/isa/riscv64/instr/rvv/vcompute.h	当前完整 helper
coding/tests/system/vdot_exact_test.S	七组系统输入和 128 位结果检查
coding/tests/system/vdot_baseline_supported.S	现有 RVV 功能程序
coding/tests/system/vdot_baseline_full1.S	基线计时片段
coding/tests/system/vdot_exact_full1.S	专用指令计时片段
coding/tests/system/vdot_empty_match1.S	空循环计时片段
coding/evidence/logs/vdot_exact_cosim_final.log	参考模型路径、Difftest enabled、S00000000
coding/evidence/logs/vdot_cosim_recheck_stopped_after_marker.log	本次复核保存记录；标记出现后停止
coding/evidence/logs/vdot_baseline_supported.log	功能输出 S0
coding/evidence/logs/vdot_baseline_full1.log	B057B
coding/evidence/logs/vdot_exact_full1.log	E00C9
coding/evidence/logs/vdot_empty_match1.log	O0094
coding/evidence/logs/nemu_exact_build.log	早期 NEMU_HOME 错误记录，不作成功构建证明
coding/evidence/waveforms/vdot_exact_wave.vcd	独立组合数据通路原始 VCD
阶段 3/video/第三阶段_视频拍摄方案与逐句旁白.docx	镜头顺序、操作和逐句口播